Interaktivt webbprojekt

Säkerhet

Sårbarhetsrisk

Inmatningsfältet `#inputMeddelande` – risk för DOM-baserad XSS. DOM-baserad XSS kan uppstå. Ingen server är inblandad, datan skulle kunna komma via ingripandens inmatning i input-fältet, JS i användarens webbläsare skulle kunna stoppa in en skadlig kod in i sidan via .innerHTML och webbläsaren tolka det som kod och manipulera DOM.

Problem fanns i koden:

```
document.getElementById("output").innerHTML = meddelande;
```

En DOM-baserad attack skickas aldrig till servern och är svåra att upptäcka av brandväggar för webbaserade program (WAF) och syns inte i serverloggar.[1]

[1] Vad är en XSS-attack? Definition och förklaring. https://www.kaspersky.se/resource-center/definitions/what-is-a-cross-site-scripting-attack (hämtad 2026-03-14)

Åtgärder

1. Först testade jag vanlig användar-/text-input i inmatningsfältet för att testa funktionen.

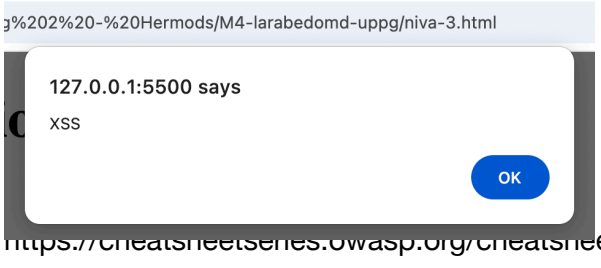
2. Därefter matade jag in:

a) `<script>alert('Du har blivit hackad!')</script>`
Chrome visar inte någon alert popp up men input via .innerHTML. Input tolkas här som HTML-kod och webbläsaren bygger DOM-element av det. Detta syns i Dev Tools:

```
    <button onclick="visaMeddelande()">Visa</button>
...  ▼<div id="output"> == $0
       <script>alert('Du har blivit hackad!')</script>
```

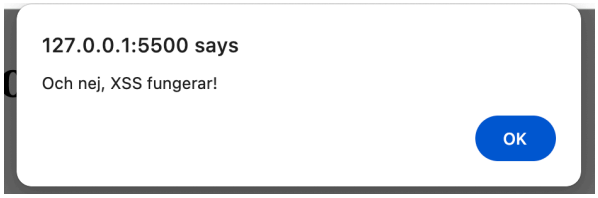
Skriptet körs inte automatiskt eftersom det finns säkerhetsskydd i moderna webbläsare.
Däremot kan HTML-attribut, som innehåller JS, med händelser som operor, onclick och oljad fortfarande triggas.

Därför testade jag också: `<IMG SRC=/ onerror="alert(String.fromCharCode(88,83,83))"></img>`[2]



https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html

och `<img src=x onerror="alert('Och nej, XSS fungerar!')">`



<img onerror>- testet fungerar som XSS. När element skapas ser webbläsaren att det är ett script-element och innehållet skickas till JS-motorn och skriptet kan aktiveras automatisktSäkerhetsåtgärd 1

I stället för innerHTML lägger jag in .textContent i JS, för då tolkas inte input som HTML utan som ren text.

document.getElementById("output").*textContent* = meddelande;

Det skapas inget DOM-element som kan trigga onerror.

Säkerhetsåtgärd 2

Ett extra skyddslager är Content Security Policy (CSP) I <head>:

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self';">
```

Denna CSP-regel kontrollerar vilka resurser såsom skript, stilar m.m. webbplatsen får ladda. Resurser som laddas får bara ha samma ursprung/domän, och resurser från externa servar är inte tillåtna som standard.

JS får bara köras från samma webbplats, extern JS blockeras. Bilder får bara laddas från samma webbplats, medan bilder från andra domäner blockeras. Stilar (CSS) får

endast ha samma ursprung som webbsidan, och eftersom styles.css ligger i samma domän som resten av webbplatsen kommer det att godkännas av CSP:n.

Men inline CSS (t.ex. `<style> body {background-color: #f5f5f5; } </style>` och inline style-attribut: (t.ex. `<div style="background-color: #f5f5f5;">`

Detta kan i vissa fall få annat betydelse, exempelvis att en extern font behöver laddas ned först i stället för att länkas till.

CSP skyddar mot säkerhetshot som Cross-Site Scripting (där angripare försöker initierar skadlig kod), data-stöld (genom att begränsa vilka externa tjänster kan nås), malware-distribution (genom att hindra från att farlig kod laddas in).

När jag bytte från innerHTML till textContent visades användarens inmatning fortfarande på sidan som vanlig text, men utan att tolkas som HTML. Det minskade risken för DOM-baserad XSS. När jag därefter lade till en Content Security Policy slutade knappen först att fungera, eftersom min kod använde inline-JavaScript i både onclick och ett <script>-block. CSP blockerade alltså denna typ av skriptkörning. För att få både funktion och säkerhet behöver JavaScript därför flyttas till en extern fil och händelsen kopplas med addEventListener. Detta ger ett bättre skydd i flera lager.

Säkerhetsåtgärd 3

Som ytterligare skydd kan inmatningen valideras eller filtreras innan den visas, exempelvis genom att ersätta tecken som “<“ och “>” i JS-koden så att de inte kan tolkas som kod. Användarens input visas redan som text, så sanitize-funktionen är mer ett extra skyddslager.

Säkerheten bygger alltså på flera lager: säker DOM-hantering, begränsning av resurser via CSP och försiktig hantering av användarinmatning.

Tillgänglighet

Navigering via <tab>

Man knappar oftast mellan länkar, knappar, formulärfält och andra interaktiva element. På den här sidan går det att tabba mellan URL-fältet, inputfältet och knappen. Sidan bör använda semantiskt HTML, såsom <button>, <section> med flera, och jag har lagt in flera <html> element som förslag, vilka är att föredra framför icke-semanstisk HTML element t.ex. .

`Fokus -`

Personer som använder skärmsläkare kan ofta hoppa mellan landmarks, regioner, rubriker. När en sektion är märkt med aria-label så blir navigationen tydligare. Knappen av mig märkt med id="button-m" markerade jag först med `aria-label` för skärmläsare, vartefter jag tog bort markeringen eftersom jag bedömde att knappen redan har ett namn genom sin synliga text "Skicka", vilket är tillräckligt uttrycksfullt för en skärmläsare.

Tabindex="0" bör användas när ett ickelement behöver förtydligas ytterligare. Att lägga in på sidan och tabindex ="0" hade dock varit överflödigt sett mot sidans nuvarande innehåll.

Bilden på sidan har fått en alt-beskrivning.

Kontrast mellan svart text och vit bakgrund är ….

Radavstånd mellan linjer bör vara minst 1.5 (satt i <body> i CSS).

Hållbarhet

Tog bort energikrävande animation som i det här fallet inte fyllde någon annan en eventuell estetisk funktion, som stal för mycket uppmärksamhet från syftet med den här sidan.

Bilder står ofta för den största datamängden på en webbsida. Jag laddade ned en bild med skrivmaskin på ett skrivbord i mediumformat (800 x 450) på 81KB och omvandlade denna via sqoosh.app till WebP för att minska filstorleken. Med Quality minskad till 75% blev och nytt filformat blev besparingen på 59% till 33KB.

Jag använde även attributet `loading="lazy"` så att bilden bara laddas när den behövs.

loading="lazy" i html innebär att bilden laddas först efter det att html har laddats ned, vilket är bra för sidans prestanda, det vill säga nedladdningstid **(men sparar inte på energi?)**

En annan åtgärd för att minska energiförbrukningen är att minimera mängden data som skickas över nätverket. Detta kan göras genom minifiering av HTML, CSS och JavaScript. Vid minifiering tas onödiga mellanslag, radbrytningar och kommentarer bort från koden, vilket gör filerna mindre. Mindre filer innebär att mindre data behöver överföras mellan server och användare, vilket minskar både laddningstid och energiförbrukning.

(*I den här uppgiften minimiserade jag dock inte koden så att den skulle synas bättre för läraren.)*

Ett annat förslag för att minska miljöpåverkan är att sidan läggs på en server

som drivs av göra energi (grön hosting).

Kortfattad reflektion om säkerhet, etik och hållbarhet.

I arbetet med webbplatsen försökte jag balansera säkerhet, tillgänglighet och hållbarhet.
Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Därför använder formuläret endast ett enkelt meddelandefält och lagrar ingen data. Jag valde även en bild utan personer för att undvika att personuppgifter eller identifierbara individer förekommer.
Jag tog även hänsyn till hållbarhet genom att optimera bilder och ta bort onödig animation som skulle kunna öka energiförbrukningen.
Jag valde en bild utan personer eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem.